O uso de agentes de LLMs para melhoramento genético de código

1st Domini da Silva Acco
DIN
UEM
Maringá, Brasil
email address or ORCID

2nd Lucas Sarri de Mello
DIN
UEM
Maringá, Brasil
pg407439@uem.br

I. Introdução

Este estudo trata-se de um experimento que avalia as mudanças na ferramenta, como elitismo, tratamento de restrição e o uso do algoritmo R-NSGA-III. O experimento foi feito utilizando a ferramenta disponibilizaa

II. Background

A. Search-Based Software Engineering (SBSE)

Search-Based Software Engineering (SBSE) consiste no uso de técnicas de busca e otimização para resolver problemas de Engenharia de Software (ES) [**?**]. A ideia central é reformular um problema de software como um problema de busca, no qual se define um espaço de possíveis soluções, uma representação adequada dessas soluções e uma função de fitness (ou função objetivo) capaz de avaliar a qualidade de cada solução candidata.

O processo típico envolve a geração iterativa de soluções candidatas, sua avaliação por meio da função de fitness e a aplicação de operadores de busca (como mutação e crossover) para explorar o espaço de soluções. SBSE tem sido aplicada em diversas áreas, incluindo teste de software, refatoração, alocação de recursos e otimização de propriedades não funcionais. O Genetic Improvement (GI) é uma das linhas de pesquisa dentro de SBSE que trata especificamente da melhoria automatizada de programas existentes.

B. Genetic Improvement (GI)

A GI é uma abordagem de SBSE que trata o código-fonte como material genético e aplica operadores evolutivos para melhorar automaticamente propriedades do software [**?**], [**?**], [**?**], [**?**]. Diferentemente de outras técnicas de otimização, o GI parte de um programa já existente e busca versões aprimoradas por meio de modificações sucessivas.

O ciclo básico do GI envolve as seguintes etapas: (1) partida de um programa original; (2) geração de variantes por meio de mutações e/ou crossover; (3) compilação das variantes; (4) execução contra um conjunto de testes; (5) avaliação da função de fitness; e (6) seleção das melhores variantes para compor a próxima geração. A função de fitness pode considerar uma ou mais propriedades, como tempo de execução, consumo de energia, uso de memória ou taxa de sucesso em testes.

Um exemplo clássico dessa abordagem é o trabalho de Bruce et al. [**?**], que aplicou GI para reduzir o consumo de energia do solver MiniSAT. Os experimentos demonstraram que era possível obter reduções de até 25% no consumo energético ao especializar o código para diferentes aplicações downstream. Esse resultado evidenciou o potencial do GI para otimizar propriedades não funcionais de difícil ajuste manual. Além disso, os autores observaram forte correlação entre consumo de energia e tempo de execução para aplicações CPU-bound e single-threaded, sugerindo que otimizar tempo pode reduzir energia [**?**].

Tradicionalmente, os operadores de mutação utilizados em GI atuam predominantemente em nível sintático, alterando statements, expressões ou nós da Abstract Syntax Tree (AST) [**?**]. Embora eficazes em diversos cenários, essas mutações tendem a gerar um grande número de variantes inválidas ou semanticamente pouco promissoras, o que aumenta o custo computacional do processo de busca [**?**].

C. Large Language Models aplicados à ES

Large Language Models (LLMs) são modelos de linguagem treinados em grandes volumes de dados textuais e de código, capazes de compreender e gerar tanto linguagem natural quanto código-fonte. No contexto da ES, LLMs têm sido empregados em tarefas como geração de código, refatoração, correção de bugs, explicação de programas e tradução entre linguagens de programação [**?**], [**?**].

Uma característica relevante dos LLMs é a capacidade de realizar transformações que consideram o contexto e a semântica do código, ao contrário de operadores puramente sintáticos. Essa propriedade os torna candidatos naturais para atuar como operadores de mutação e crossover mais inteligentes em abordagens de GI, tema que será discutido nas seções seguintes.

III. Limitações do GI Tradicional

Apesar dos resultados promissores obtidos com GI, a abordagem tradicional apresenta limitações importantes que restringem sua eficácia e eficiência, especialmente quando aplicada a programas complexos ou quando se busca otimizações mais sofisticadas.

Uma das principais limitações está relacionada à natureza dos operadores de mutação comumente utilizados. Na maioria

das implementações clássicas de GI, as mutações operam predominantemente em nível sintático, alterando statements, expressões ou nós da Abstract Syntax Tree (AST) de forma relativamente cega [**?**], [**?**]. Embora essas transformações sejam capazes de explorar o espaço de busca, elas frequentemente desconsideram o contexto semântico do código e as dependências entre diferentes partes do programa. Como consequência, uma parcela significativa das variantes geradas resulta em código que não compila ou que falha nos testes.

O grande volume de variantes inválidas impacta diretamente o custo computacional do processo. Cada candidata precisa ser compilada e executada contra a suíte de testes para que sua aptidão (fitness) seja avaliada. Quando a taxa de mutações válidas é baixa, a maior parte do esforço computacional é gasta avaliando soluções que não contribuem para o progresso da busca [**?**], [**?**].

Além disso, os operadores tradicionais de GI tendem a ser pouco informativos. Eles não aproveitam conhecimento prévio sobre padrões de otimização ou sobre a intenção por trás de determinadas estruturas de código. Isso faz com que a exploração do espaço de busca seja, em grande parte, aleatória ou guiada apenas por heurísticas genéricas, o que pode tornar a convergência mais lenta, especialmente em programas de maior porte.

Por fim, a eficácia do GI depende fortemente da qualidade do oráculo de teste. Como a função de fitness se baseia na execução dos testes, a ausência de uma suíte de testes robusta e representativa pode levar a otimizações enganosas ou à aceitação de variantes que alteram o comportamento esperado do programa em cenários não cobertos.

Essas limitações motivam a investigação de operadores de mutação e crossover mais inteligentes, capazes de considerar o contexto e a semântica do código. Nesse sentido, os LLMs surgem como uma alternativa promissora para aprimorar o processo de GI, tema discutido na seção seguinte.

IV. LLM como Operador Genético

Antes de apresentar o GI-Agent, é importante destacar evidências de que LLMs já melhoram operadores individuais do Genetic Improvement.

Brownlee et al. [**?**] avaliaram o uso de LLMs como operadores de mutação. Os resultados mostraram que edições geradas por LLM passaram nos testes em 77% dos casos, taxa significativamente superior à das mutações tradicionais. Embora o número de edições únicas tenha sido menor, a qualidade das variantes válidas foi superior.

Em outra direção, Bouras et al. [**?**] investigaram o uso de LLMs no operador de crossover. A abordagem assistida por LLM obteve o melhor ranking médio entre os métodos avaliados e produziu uma melhoria de 8,5% no fitness (tempo de execução), além de reduzir em 25,6% a quantidade de variantes necessárias para atingir os mesmos patamares de desempenho.

Esses resultados mostram que LLMs podem atuar de forma eficaz tanto na mutação quanto no crossover, preparando o terreno para abordagens mais completas baseadas em agentes, como o GI-Agent.

V. Genetic Improvement Agent: GI-Agent

[**?**] apresentam que o GI pode ser aplicado com sucesso à otimização automatizada de programas, incluindo a redução do consumo de energia e a otimização da eficiência em tempo de execução, entre outras aplicações. Os resultados apresentados também evidenciam o potencial do uso de LLMs para orientar operações de melhoramento de software. Nesse contexto, [**?**] propõem o GI-Agent, uma abordagem que integra LLMs ao GI para realizar a otimização autônoma do código-fonte e superar algumas limitações das abordagens tradicionais. A proposta estende o trabalho de [**?**] por meio da incorporação de mecanismos de aprendizado evolutivo multigeracional, memória de ações passadas e raciocínio orientado por contexto.

O GI-Agent utiliza um LLM para orientar o processo evolutivo a partir de informações obtidas em gerações anteriores. Essas informações são armazenadas na forma de *reflexões*, que registram insights sobre os sucessos e falhas observados durante o processo de compilação e execução do software. Dessa forma, as reflexões são utilizadas para refinar progressivamente as estratégias empregadas nas mutações e cruzamentos, tornando as alterações menos estocásticas e mais sensíveis ao contexto do programa.

A abordagem apresenta três características principais:

- **Operações genéticas sensíveis ao contexto:** o GI tradicional aplica operadores de mutação e cruzamento de forma predominantemente heurística. O GI-Agent utiliza LLMs para realizar edições considerando o contexto do código, bem como os resultados obtidos em edições anteriores, permitindo selecionar alterações de maneira mais informada.
- **Aprendizado iterativo entre gerações:** embora o GI tradicional seja orientado por pontuações de aptidão, ele não possui capacidade explícita de registrar e generalizar informações provenientes de edições anteriores. O GI-Agent introduz um mecanismo de reflexão que armazena o feedback obtido a cada edição, incluindo informações relacionadas a sucessos e falhas em tempo de compilação e execução. Essas reflexões são posteriormente utilizadas para orientar e refinar as decisões de mutação e cruzamento nas gerações seguintes.
- **Interpretabilidade e justificativa:** no GI tradicional, os *patches* são gerados por meio de processos aleatórios ou heurísticos, não fornecendo explicações sobre as razões associadas a uma determinada alteração. No GI-Agent, o LLM é solicitado a elaborar uma estratégia antes de produzir uma edição. Esse raciocínio é incorporado ao processo como uma reflexão revisável, fornecendo informações sobre a intenção associada às modificações realizadas.

O GI-Agent foi integrado ao *framework* Magpie GI e avaliado utilizando os benchmarks SAT4J, para programas Java, e MiniSAT, para programas C++. Os resultados apresentados por [**?**] indicam que a abordagem foi capaz de

gerar consistentemente variantes mais viáveis e com melhor desempenho em comparação ao processo tradicional. Além disso, a combinação de *few-shot prompting* com uma estratégia de busca estruturada demonstra uma forma de utilizar LLMs para orientar a exploração do espaço de possíveis modificações e aprimorar o processo de otimização automatizada de programas.

VI. Discussão e Limitações

A integração de LLMs ao Genetic Improvement traz ganhos claros em viabilidade e qualidade das variantes. No entanto, algumas limitações permanecem.

O custo computacional da inferência de LLMs ainda é elevado, especialmente em loops evolutivos longos, o que pode limitar a aplicação em larga escala. Além disso, os LLMs tendem a gerar edições semanticamente semelhantes entre si, o que pode reduzir a diversidade da busca [**?**]. Essa característica foi observada também nos experimentos do GI-Agent, nos quais os métodos com reflexão apresentaram maior taxa de patches repetidos.

A eficácia das abordagens depende fortemente da qualidade da suíte de testes e da formulação dos prompts. Resultados reportados na literatura indicam que prompts mais simples frequentemente performam melhor do que prompts complexos com muitos exemplos. Outro ponto relevante é a ausência de garantias formais de preservação de comportamento. Embora os testes funcionem como oráculo, não há ainda mecanismos que assegurem semanticamente a corretude das otimizações em todos os cenários.

VII. Conclusão

Este trabalho discutiu o uso de agentes baseados em Large Language Models para aprimorar o processo de Genetic Improvement. Foi apresentado como um dos exemplo o GI-Agent, uma abordagem que combina LLMs com mecanismos de reflexão para tornar as operações de mutação e crossover mais contextualizadas e menos estocásticas.

Os resultados disponíveis na literatura indicam que o uso de LLMs aumenta significativamente a taxa de variantes viáveis, reduz erros de compilação e melhora a eficiência da busca. Evidências adicionais mostram ganhos também quando LLMs são utilizados isoladamente como operadores de mutação e crossover.

Apesar dos desafios restantes, como custo computacional, menor diversidade e dependência de oráculos de teste, a combinação entre busca evolutiva e modelos de linguagem representa um caminho promissor para a otimização automatizada de código. Trabalhos futuros podem explorar o uso de modelos menores e mais eficientes, além de mecanismos para aumentar a diversidade sem perder a qualidade das variantes geradas.

References

[1] J. Castanho, C. G. Britez, W. M. Freire, A. M. M. M. Amaral, T. E. Colanzi, and W. K. G. Assunção, "Human in the loop for microservice identification: An exploratory study," in *Proc. 2026 Brazilian Symposium on Software Components, Architectures, and Reuse (SBCARS)*, São Paulo, Brazil, 2026. [Online]. Available: https://cbsoft.sbc.org.br/2026/data/papers/sbcars/Human%20in%20the%20loop%20for%20microservice%20identification%20An%20exploratory%20study.pdf. Accessed: Nov. 4, 2026.